

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра Информационных систем

ОТЧЁТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №2
по дисциплине «Методы и средства проектирования информационных
систем»

Тема: Приемы моделирования Перечислений

Студенты:

Группа	ФИО	Подпись исп.	Оценка	Подпись преп.
3376	Карпенко Д.В.			
3376	Хаджикулов А.			
3376	Хуснутдинов Р.Р.			

Преподаватель

Дубенецкий В.А.

Санкт-Петербург

2026

АННОТАЦИЯ

В работе представлено расширение классификатора продукции LEGO, обеспечивающее поддержку перечисляемых типов атрибутов (Enumeration). Разработаны модели классов этапа проектирования и хранения данных для динамического создания перечислений и их значений. Реализован набор SQL-процедур и REST API-эндпоинтов для управления перечислениями: создание, редактирование, изменение порядка значений, удаление. Выполнено тестирование разработанных процедур с примерами перечислений «Тип детали», «Цвет детали», «Редкость мини-фигурки».

SUMMARY

The paper presents an extension of the LEGO product classifier that provides support for enumerated attribute types (Enumeration). Models of design stage classes and data storage have been developed for dynamic creation of enumerations and their values. A set of SQL procedures and REST API endpoints has been implemented for managing enumerations: creating, editing, reordering values, and deleting. The developed procedures were tested with examples of the enumerations "Part type", "Part color", and "Rarity of the mini-figure".

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ.....	5
2. РАЗРАБОТКА ФУНКЦИОНАЛЬНЫХ ТРЕБОВАНИЙ	6
3. РАЗРАБОТКА ОБЪЕКТНОЙ МОДЕЛИ ЭТАПА ПРОЕКТИРОВАНИЯ ..	8
4. РАЗРАБОТКА МОДЕЛИ ХРАНЕНИЯ (ER-ДИАГРАММА)	9
5. РАЗРАБОТКА ОСНОВНЫХ SQL-ПРОЦЕДУР	10
6. ТЕСТИРОВАНИЕ РАЗРАБОТАННЫХ ПРОЦЕДУР.....	11
ЗАКЛЮЧЕНИЕ	15
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	16
ПРИЛОЖЕНИЕ А	17
ПРИЛОЖЕНИЕ Б.....	26

ВВЕДЕНИЕ

В информационных системах часто встречаются атрибуты, принимающие значения из заранее известного фиксированного набора – перечисления (enum). Примеры: тип детали («Кирпич», «Плита»), цвет («Красный», «Синий»), редкость («Common», «Rare»). В классическом подходе такие перечисления жёстко зашиваются в код приложения, что затрудняет их изменение без перекомпиляции.

Целью данной работы является расширение существующего классификатора продукции LEGO поддержкой динамических перечислений – таких, которые можно создавать, редактировать и удалять через API без изменения программного кода.

Задачи:

1. Проанализировать предметную область LEGO и выделить атрибуты, требующие реализации в виде перечислений.
2. Разработать диаграмму вариантов использования для подсистемы управления перечислениями.
3. Спроектировать модель классов этапа проектирования.
4. Разработать ER-диаграмму для хранения перечислений в реляционной БД.
5. Реализовать SQL-процедуры и API-эндпоинты для работы с перечислениями.
6. Выполнить тестирование и задокументировать результаты.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

В существующем классификаторе продукции LEGO были выявлены следующие атрибуты, которые целесообразно реализовать в виде перечислений. Атрибуты, реализованные в виде перечислений представлены в таблице 1.

Таблица 1 – Реализованные перечисления атрибутов

Атрибут	Возможные значения	Обоснование
Тип детали	Кирпич, Плита, Техническая, Специальная	Значения фиксированы, но могут расширяться (например, «Декоративная»)
Цвет детали	Красный, Синий, Зелёный, Белый, Чёрный	Цвета стандартизированы, но компания LEGO периодически вводит новые
Редкость мини-фигурки	Common, Rare, Exclusive	Используется для оценки коллекционной ценности

Требование к перечислениям:

Они должны быть динамическими – администратор системы должен иметь возможность добавлять новые значения, изменять порядок вывода и удалять устаревшие значения через API, без остановки работы системы и без изменения кода.

2 РАЗРАБОТКА ФУНКЦИОНАЛЬНЫХ ТРЕБОВАНИЙ

На основе анализа была разработана диаграмма вариантов использования для подсистемы управления перечислениями. Диаграмма представлена на рисунке 1.

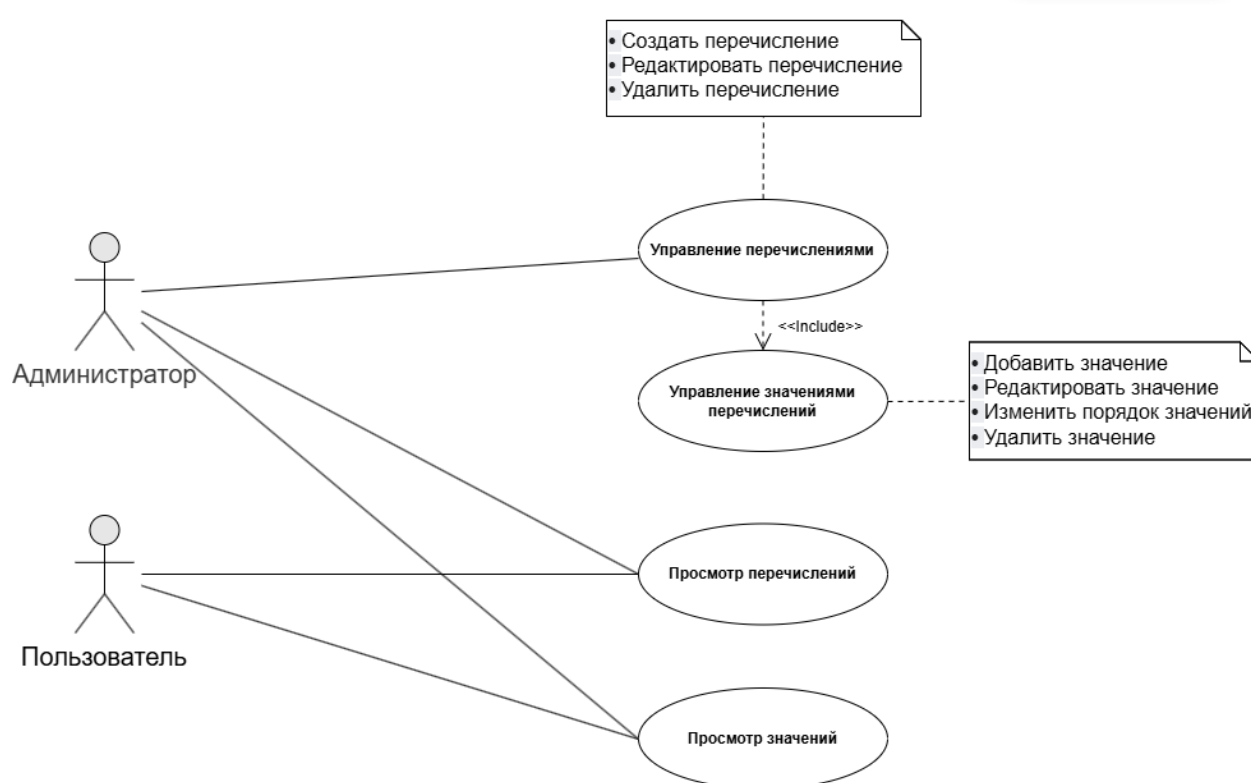


Рисунок 1 – Диаграмма вариантов использования для подсистемы «Перечисления»

Актёры:

- Администратор — управляет перечислениями и их значениями.
- Пользователь — только просматривает перечисления и значения.

Варианты использования (прецеденты):

Для Администратора:

1. Управление перечислениями:

- Создать перечисление
- Редактировать перечисление
- Удалить перечисление

2. Управление значениями перечислений:

- Добавить значение
- Редактировать значение
- Изменить порядок значений
- Удалить значение

Для Пользователя:

1. Просмотр перечислений – получить список всех перечислений.

2. Просмотр значений – получить значения конкретного перечисления в заданном порядке.

Связи:

- Администратор наследует все прецеденты Пользователя.
- Прецедент «Управление перечислениями» включает «Управление значениями перечислений» (так как перечисление без значений не имеет смысла).

3 РАЗРАБОТКА ОБЪЕКТНОЙ МОДЕЛИ ЭТАПА ПРОЕКТИРОВАНИЯ

На основе паттерна «Классификатор» была разработана диаграмма классов для работы с перечислениями. Диаграмма представлена на рисунке 2.

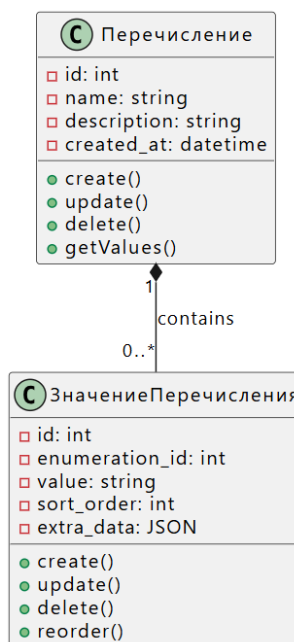


Рисунок 2 – Диаграмма классов для подсистемы «Перечисления»

Описание компонентов диаграммы:

- Метакласс «Перечисление» – хранит информацию о типе перечисления (например, «Тип детали»). Атрибут `created_at` фиксирует дату создания.
- Класс «ЗначениеПеречисления» – хранит конкретные значения (например, «Кирпич»). Атрибут `sort_order` определяет порядок вывода. Поле `extra_data` (JSON) предусмотрено для хранения дополнительных метаданных (иконка, цвет в интерфейсе и т.п.).
- Связь «композиция» (закрашенный ромб) означает, что значение не может существовать без родительского перечисления. При удалении перечисления все его значения удаляются автоматически (каскадно).

4 РАЗРАБОТКА МОДЕЛИ ХРАНЕНИЯ (ER-ДИАГРАММА)

Для реализации объектной модели в реляционной СУБД PostgreSQL разработана ER-диаграмма. Диаграмма представлена на рисунке 3.

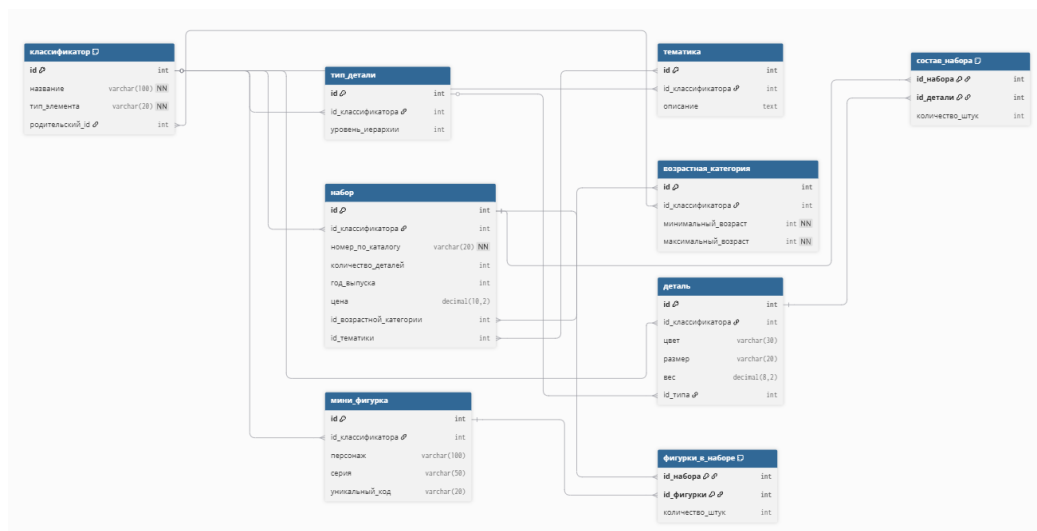


Рисунок 3 – ER-диаграмма для подсистемы «Перечисления»

В таблице 2 представлено описание сущностей и связей ER-диаграммы.

Таблица 2 – Описание сущностей и связей

Сущность	Назначение	Атрибуты
перечисление	Хранит типы перечислений	id (PK), name (уникальное имя), description, created_at
значение_перечисления	Хранит значения перечислений	id (PK), enumeration_id (FK), value, sort_order, extra_data

Связь: «один-ко-многим» между перечисление и значение_перечисления. Реализована через внешний ключ enumeration_id с каскадным удалением (ON DELETE CASCADE).

5 РАЗРАБОТКА ОСНОВНЫХ SQL-ПРОЦЕДУР

В рамках серверной части на FastAPI были реализованы следующие методы класса LegoClassifier для работы с перечислениями (полный код приведён в Приложении А).

В таблице 3 представлены основные процедуры, реализованные для работы с перечислениями.

Таблица 3 – Основные процедуры и их реализации

Процедура	Реализация
Формирование классификатора перечислений	Таблица перечисление
Создание нового перечисления	POST /enumerations
Редактирование списка значений	PUT /enum-values/{value_id}
Изменение порядка позиций	PUT /enumerations/{enum_id}/values/reorder
Вывод значений перечисления	GET /enumerations/{enum_id}/values
Вывод перечислений	GET /enumerations
Выбор значения перечисления	GET /enumerations/{enum_id}/values (клиент выбирает нужное)

6 ТЕСТИРОВАНИЕ РАЗРАБОТАННЫХ ПРОЦЕДУР

Тестирование выполнялось через Swagger UI (<http://localhost:8000/docs>) после очистки базы и загрузки тестовых данных.

6.1. Запуск сервера и проверка готовности

Перед началом тестирования необходимо запустить серверное приложение. На рисунке 4 представлен успешный запуск сервера FastAPI.

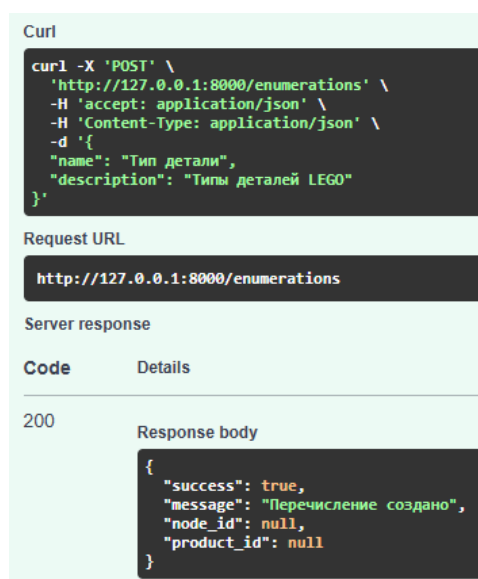
```
2026-04-14 00:42:13,562 INFO sqlalchemy.engine.Engine [generated in 0.00054s] ()
2026-04-14 00:42:13,568 INFO sqlalchemy.engine.Engine ROLLBACK
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

Рисунок 4 – Запуск сервера FastAPI

6.2. Создание перечисления

Тест 1.1: Создание перечисления «Тип детали»

Перейдём в раздел POST /enumerations и нажмём Try it out, затем Execute. Попробуем создать перечисление «Тип детали». Результат создания перечисления представлен на рисунке 5.



The screenshot shows the Swagger UI interface for a POST request to the endpoint `http://127.0.0.1:8000/enumerations`. The request body is a JSON object: `{ "name": "Тип детали", "description": "Типы деталей LEGO" }`. The server response is a 200 status code with a JSON body: `{ "success": true, "message": "Перечисление создано", "node_id": null, "product_id": null }`.

```
Curl
curl -X 'POST' \
  'http://127.0.0.1:8000/enumerations' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Тип детали",
    "description": "Типы деталей LEGO"
  }'

Request URL
http://127.0.0.1:8000/enumerations

Server response
Code    Details
200
Response body
{
  "success": true,
  "message": "Перечисление создано",
  "node_id": null,
  "product_id": null
}
```

Рисунок 5 – Создание перечисления

6.3. Добавление значений

Тест 2.1: Добавление значения «Кирпич»

Перейдём в раздел POST /enumerations/1/values и нажмём Try it out, затем Execute. Попробуем добавить в «Тип детали» значение «Кирпич». Результат добавления представлен на рисунке 6.

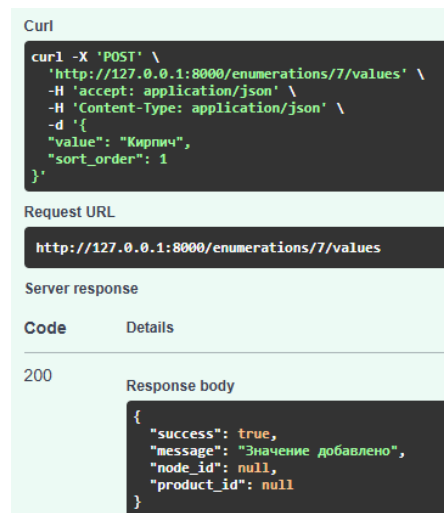


Рисунок 6 – Добавление значения «Кирпич»

6.4. Получение списка перечислений

Тест 3.1: Получение всех перечислений

Перейдём в раздел GET /enumerations и нажмём Try it out, затем Execute. Результат получения списка перечислений представлен на рисунке 7.

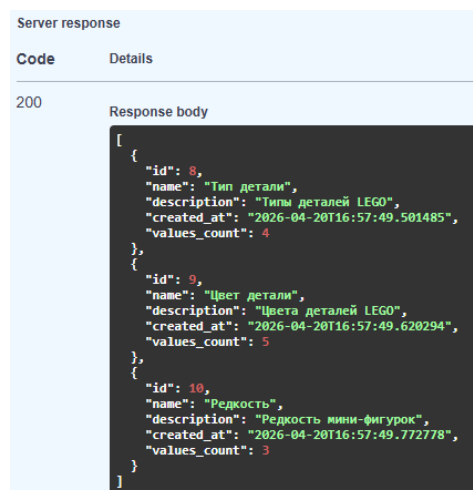


Рисунок 7 – Список перечислений

6.5. Получение значений перечисления

Тест 3.1: Получение значений «Тип детали»

Перейдём в раздел GET /enumerations/4/values и нажмём Try it out, затем Execute. Результат получения значений представлен на рисунке 8.



Рисунок 8 – Значения перечисления

6.6. Изменение порядка значений

Тест 5.1: Изменение порядка значений

Перейдём в раздел PUT /enumerations/4/values/reorder и нажмём Try it out, введём новый порядок значений, затем Execute. Результат изменения порядка значений представлен на рисунке 9.

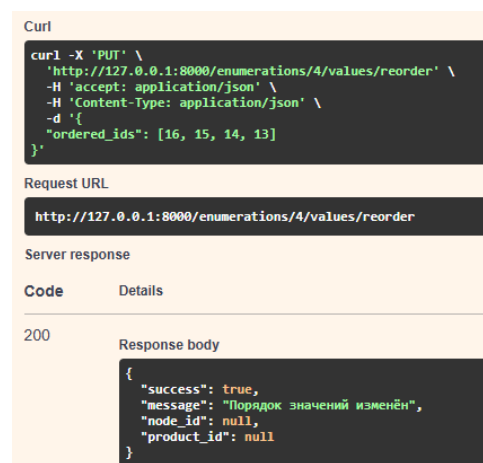


Рисунок 9 – Изменение порядка значений

6.7. Удаление значения

Тест 6.1: Удаление значения «Техническая»

Перейдём в раздел DELETE /enum-values/15 и нажмём Try it out, введём 15, затем Execute. Результат удаления значения представлен на рисунке 10.

Request URL	
<code>http://127.0.0.1:8000/enum-values/15</code>	
Server response	
Code	Details
200	Response body <pre>{ "success": true, "message": "Значение удалено", "node_id": null, "product_id": null }</pre>

Рисунок 10 – Удаление значения

6.8. Удаление перечисления

Тест 7.1: Удаление перечисления «Редкость»

Перейдём в раздел DELETE /enumerations/6 и нажмём Try it out, введём 6, затем Execute. Результат удаления значения представлен на рисунке 11.

Request URL	
<code>http://127.0.0.1:8000/enumerations/6</code>	
Server response	
Code	Details
200	Response body <pre>{ "success": true, "message": "Перечисление удалено", "node_id": null, "product_id": null }</pre>

Рисунок 11 – Удаление перечисления

ЗАКЛЮЧЕНИЕ

В рамках практического задания 1.2 было выполнено расширение классификатора продукции LEGO поддержкой динамических перечислений.

Что сделано:

1. Анализ предметной области — выявлены атрибуты, требующие реализации в виде перечислений («Тип детали», «Цвет детали», «Редкость мини-фигурки»).
2. Функциональные требования — разработана диаграмма вариантов использования с разделением ролей Администратора и Пользователя.
3. Объектная модель этапа проектирования — построена диаграмма классов с метаклассом «Перечисление» и классом «ЗначениеПеречисления», связанными композицией. Реализована поддержка сортировки значений через поле `sort_order`.
4. Модель хранения — разработана ER-диаграмма для PostgreSQL, включающая таблицы `перечисление` и `значение_перечисления` с каскадным удалением.
5. SQL-процедуры — реализованы методы в классе `LegoClassifier` для полного CRUD перечислений и их значений, включая изменение порядка.
6. Тестирование — выполнено через Swagger UI; протестированы все эндпоинты, включая штатные и ошибочные сценарии (попытка создать дубликат, удалить несуществующее значение и т.п.).

Результат: Разработанная подсистема перечислений полностью соответствует функциональным требованиям задания 1.2 и может быть интегрирована в существующий классификатор LEGO. Все изменения структуры перечислений (добавление/удаление значений, изменение порядка) выполняются динамически, без остановки сервера и без изменения программного кода.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Дубенецкий В.А., Кузнецов А.Г. Проектирование информационных систем с использованием UML: учебно-методическое пособие. – СПбГЭТУ «ЛЭТИ», 2016.
2. Дубенецкий В.А., Кузнецов А.Г. Использование объектных моделей при создании информационных систем: учебно-методическое пособие. – СПбГЭТУ «ЛЭТИ», 2016.
3. Гамма Э., Хелм Р., Джонсон Р., Влиссидес Дж. Приемы объектно-ориентированного проектирования. Паттерны проектирования. – СПб: Питер, 2018.
4. Документация PostgreSQL / [Электронный ресурс] // PostgreSQL Documentation :[сайт]. – URL: <https://www.postgresql.org/docs/> (дата обращения: 02.04.2026).
5. Каталог Лего / [Электронный ресурс] // LEGO Product Catalog :[сайт]. – URL: <https://www.lego.com/> (дата обращения: 02.04.2026).

ПРИЛОЖЕНИЕ А

КОД SQLALCHEMY-МОДЕЛЕЙ

```
# Дополнения к models.py

from datetime import datetime

from sqlalchemy import Column, Integer, String, Text, DateTime, JSON,
ForeignKey

from sqlalchemy.orm import relationship

class Enumeration(Base):
    """Перечисление — набор фиксированных значений"""
    __tablename__ = "перечисление"

    id = Column(Integer, primary_key=True, autoincrement=True)
    name = Column(String(100), nullable=False, unique=True)    # "Тип детали",
    "Цвет"
    description = Column(Text)                                # Описание
    created_at = Column(DateTime, default=datetime.utcnow)    # Дата создания

    # Связь со значениями (каскадное удаление)
    values = relationship("EnumValue", back_populates="enumeration",
        cascade="all, delete-orphan")

class EnumValue(Base):
    """Значение перечисления"""
    __tablename__ = "значение_перечисления"

    id = Column(Integer, primary_key=True, autoincrement=True)
    enumeration_id = Column(Integer, ForeignKey("перечисление.id",
        ondelete="CASCADE"), nullable=False)
```

```

    value = Column(String(200), nullable=False)          # "Кирпич",
"Красный"

    sort_order = Column(Integer, default=0)              # Порядок вывода

    extra_data = Column(JSON, nullable=True)             # Запас: иконка, цвет
и т.п.

```

```

# Связь с родительским перечислением

```

```

enumeration = relationship("Enumeration", back_populates="values")

```

```

# Дополнения к lego_classifier.py

```

```

from models import Classifier, Theme, AgeCategory, PartType, Set, Part,
Minifigure, SetPart, SetMinifigure, Enumeration, EnumValue

```

```

def add_enumeration(self, db: Session, name: str, description: str = None) ->
Dict[str, Any]:

```

```

    """Создать новое перечисление"""

```

```

    exists = db.query(Enumeration).filter(Enumeration.name == name).first()

```

```

    if exists:

```

```

        return {"success": False, "message": f"Перечисление '{name}' уже
существует"}

```

```

    try:

```

```

        enum = Enumeration(name=name, description=description)

```

```

        db.add(enum)

```

```

        db.commit()

```

```

        db.refresh(enum)

```

```

        return {"success": True, "message": "Перечисление создано", "enum_id":
enum.id}

```

```

    except Exception as e:

```

```

        db.rollback()

```

```

        return {"success": False, "message": str(e)}

```

```

def get_all_enumerations(self, db: Session) -> List[Dict[str, Any]]:
    """Получить все перечисления с количеством значений"""
    enums = db.query(Enumeration).all()
    result = []
    for e in enums:
        values_count = db.query(EnumValue).filter(EnumValue.enumeration_id
== e.id).count()
        result.append({
            "id": e.id,
            "name": e.name,
            "description": e.description,
            "created_at": e.created_at,
            "values_count": values_count
        })
    return result

```

```

def get_enumeration_by_id(self, db: Session, enum_id: int) -> Dict[str, Any]:
    """Получить перечисление по ID"""
    enum = db.query(Enumeration).filter(Enumeration.id == enum_id).first()
    if not enum:
        return {"success": False, "message": "Перечисление не найдено"}
    return {
        "success": True,
        "id": enum.id,
        "name": enum.name,
        "description": enum.description,
        "created_at": enum.created_at
    }

```

```
}
```

```
def update_enumeration(self, db: Session, enum_id: int, name: str = None,
description: str = None) -> Dict[str, Any]:
```

```
    """Обновить перечисление"""
```

```
    enum = db.query(Enumeration).filter(Enumeration.id == enum_id).first()
```

```
    if not enum:
```

```
        return {"success": False, "message": "Перечисление не найдено"}
```

```
    try:
```

```
        if name is not None:
```

```
            enum.name = name
```

```
        if description is not None:
```

```
            enum.description = description
```

```
        db.commit()
```

```
        return {"success": True, "message": "Перечисление обновлено"}
```

```
    except Exception as e:
```

```
        db.rollback()
```

```
        return {"success": False, "message": str(e)}
```

```
def delete_enumeration(self, db: Session, enum_id: int) -> Dict[str, Any]:
```

```
    """Удалить перечисление (каскадно удалит все значения)"""
```

```
    enum = db.query(Enumeration).filter(Enumeration.id == enum_id).first()
```

```
    if not enum:
```

```
        return {"success": False, "message": "Перечисление не найдено"}
```

```
    try:
```

```
        db.delete(enum)
```

```
        db.commit()
```

```
        return {"success": True, "message": "Перечисление удалено"}
```

```

except Exception as e:

    db.rollback()

    return {"success": False, "message": str(e)}


# ----- Значения перечислений -----


def add_enum_value(self, db: Session, enum_id: int, value: str, sort_order: int =
None, extra_data: dict = None) -> Dict[str, Any]:

    """Добавить значение в перечисление"""

    enum = db.query(Enumeration).filter(Enumeration.id == enum_id).first()

    if not enum:

        return {"success": False, "message": "Перечисление не найдено"}

    if sort_order is None:

        max_order = db.query(func.coalesce(func.max(EnumValue.sort_order),
0)).filter(EnumValue.enumeration_id == enum_id).scalar()

        sort_order = max_order + 1

    try:

        ev = EnumValue(enumeration_id=enum_id, value=value,
sort_order=sort_order, extra_data=extra_data)

        db.add(ev)

        db.commit()

        db.refresh(ev)

        return {"success": True, "message": "Значение добавлено", "value_id":
ev.id}

    except Exception as e:

        db.rollback()

        return {"success": False, "message": str(e)}


def get_enum_values(self, db: Session, enum_id: int) -> List[Dict[str, Any]]:

```

```

        """Получить все значения перечисления (сортировка по sort_order)"""
        values = db.query(EnumValue).filter(EnumValue.enumeration_id ==
enum_id).order_by(EnumValue.sort_order).all()
        return [{
            "id": v.id,
            "enumeration_id": v.enumeration_id,
            "value": v.value,
            "sort_order": v.sort_order,
            "extra_data": v.extra_data
        } for v in values]

```

```

def update_enum_value(self, db: Session, value_id: int, value: str = None,
sort_order: int = None, extra_data: dict = None) -> Dict[str, Any]:

```

```

        """Обновить значение перечисления"""

```

```

        ev = db.query(EnumValue).filter(EnumValue.id == value_id).first()

```

```

        if not ev:

```

```

            return {"success": False, "message": "Значение не найдено"}

```

```

        try:

```

```

            if value is not None:

```

```

                ev.value = value

```

```

            if sort_order is not None:

```

```

                ev.sort_order = sort_order

```

```

            if extra_data is not None:

```

```

                ev.extra_data = extra_data

```

```

            db.commit()

```

```

            return {"success": True, "message": "Значение обновлено"}

```

```

        except Exception as e:

```

```

            db.rollback()

```

```

        return {"success": False, "message": str(e)}

def reorder_enum_values(self, db: Session, enum_id: int, ordered_ids: List[int])
-> Dict[str, Any]:
    """Изменить порядок значений перечисления"""
    # Проверка, что все ID принадлежат этому перечислению
    for vid in ordered_ids:
        ev = db.query(EnumValue).filter(EnumValue.id == vid,
EnumValue.enumeration_id == enum_id).first()
        if not ev:
            return {"success": False, "message": f"Значение {vid} не принадлежит
перечислению {enum_id}"}
        try:
            for idx, vid in enumerate(ordered_ids, start=1):
                db.query(EnumValue).filter(EnumValue.id ==
vid).update({"sort_order": idx})
            db.commit()
            return {"success": True, "message": "Порядок значений изменён"}
        except Exception as e:
            db.rollback()
            return {"success": False, "message": str(e)}

def delete_enum_value(self, db: Session, value_id: int) -> Dict[str, Any]:
    """Удалить значение перечисления"""
    ev = db.query(EnumValue).filter(EnumValue.id == value_id).first()
    if not ev:
        return {"success": False, "message": "Значение не найдено"}
    try:
        db.delete(ev)

```

```

        db.commit()

        return {"success": True, "message": "Значение удалено"}

except Exception as e:

    db.rollback()

    return {"success": False, "message": str(e)}

# Добавить тестовые перечисления в метод load_test_data

print(" Добавляем тестовые перечисления...")

# Перечисление "Тип детали"

enum_part_type = self.add_enumeration(db, "Тип детали", "Типы деталей
LEGO")

if enum_part_type["success"]:

    enum_id = enum_part_type["enum_id"]

    self.add_enum_value(db, enum_id, "Кирпич", 1)

    self.add_enum_value(db, enum_id, "Плита", 2)

    self.add_enum_value(db, enum_id, "Техническая", 3)

    self.add_enum_value(db, enum_id, "Специальная", 4)

# Перечисление "Цвет детали"

enum_color = self.add_enumeration(db, "Цвет детали", "Цвета деталей
LEGO")

if enum_color["success"]:

    enum_id = enum_color["enum_id"]

    self.add_enum_value(db, enum_id, "Красный", 1)

    self.add_enum_value(db, enum_id, "Синий", 2)

    self.add_enum_value(db, enum_id, "Зелёный", 3)

    self.add_enum_value(db, enum_id, "Белый", 4)

```



```
self.add_enum_value(db, enum_id, "Чёрный", 5)

# Перечисление "Редкость мини-фигурки"
enum_rarity = self.add_enumeration(db, "Редкость", "Редкость мини-
фигурок")
if enum_rarity["success"]:
    enum_id = enum_rarity["enum_id"]
    self.add_enum_value(db, enum_id, "Common", 1)
    self.add_enum_value(db, enum_id, "Rare", 2)
    self.add_enum_value(db, enum_id, "Exclusive", 3)

db.commit()

print(" Тестовые перечисления добавлены")
```

ПРИЛОЖЕНИЕ Б

СКРИПТ ДЛЯ ЗАПУСКА СЕРВЕРА FASTAPI

```
# Дополнения к schemas.py
from datetime import datetime

class EnumerationCreate(BaseModel):
    name: str
    description: Optional[str] = None

class EnumerationResponse(BaseModel):
    id: int
    name: str
    description: Optional[str] = None
    created_at: datetime
    values_count: Optional[int] = 0

class Config:
    from_attributes = True

# EnumValue schemas
class EnumValueCreate(BaseModel):
    value: str
    sort_order: Optional[int] = None
    extra_data: Optional[Dict[str, Any]] = None

class EnumValueResponse(BaseModel):
    id: int
    enumeration_id: int
    value: str
    sort_order: int
```

```
extra_data: Optional[Dict[str, Any]] = None
```

```
class Config:
```

```
    from_attributes = True
```

```
class EnumValueReorder(BaseModel):
```

```
    ordered_ids: List[int]
```

```
# Дополнения к main.py (эндпоинты)
```

```
@app.post("/enumerations", response_model=OperationResult)
```

```
def create_enumeration(data: EnumerationCreate, db: Session = Depends(get_db)):
```

```
    """Создать новое перечисление"""
```

```
    result = classifier.add_enumeration(db, data.name, data.description)
```

```
    if not result["success"]:
```

```
        raise HTTPException(status_code=400, detail=result["message"])
```

```
    return result
```

```
@app.get("/enumerations", response_model=List[EnumerationResponse])
```

```
def get_enumerations(db: Session = Depends(get_db)):
```

```
    """Получить все перечисления"""
```

```
    return classifier.get_all_enumerations(db)
```

```
@app.get("/enumerations/{enum_id}", response_model=EnumerationResponse)
```

```
def get_enumeration(enum_id: int, db: Session = Depends(get_db)):
```

```
    """Получить перечисление по ID"""
```

```
    result = classifier.get_enumeration_by_id(db, enum_id)
```

```
    if not result["success"]:
```

```
        raise HTTPException(status_code=404, detail=result["message"])
```

```
    return result
```

```

@app.put("/enumerations/{enum_id}", response_model=OperationResult)
def update_enumeration(enum_id: int, data: EnumerationCreate, db: Session =
Depends(get_db)):
    """Обновить перечисление"""
    result = classifier.update_enumeration(db, enum_id, data.name,
data.description)
    if not result["success"]:
        raise HTTPException(status_code=400, detail=result["message"])
    return result

```

```

@app.delete("/enumerations/{enum_id}", response_model=OperationResult)
def delete_enumeration(enum_id: int, db: Session = Depends(get_db)):
    """Удалить перечисление"""
    result = classifier.delete_enumeration(db, enum_id)
    if not result["success"]:
        raise HTTPException(status_code=400, detail=result["message"])
    return result

```

----- Значения перечислений -----

```

@app.post("/enumerations/{enum_id}/values", response_model=OperationResult)
def create_enum_value(enum_id: int, data: EnumValueCreate, db: Session =
Depends(get_db)):
    """Добавить значение в перечисление"""
    result = classifier.add_enum_value(db, enum_id, data.value, data.sort_order,
data.extra_data)
    if not result["success"]:
        raise HTTPException(status_code=400, detail=result["message"])
    return result

```

```

@app.get("/enumerations/{enum_id}/values",
response_model=List[EnumValueResponse])
def get_enum_values(enum_id: int, db: Session = Depends(get_db)):
    """Получить все значения перечисления"""
    return classifier.get_enum_values(db, enum_id)

@app.put("/enum-values/{value_id}", response_model=OperationResult)
def update_enum_value(value_id: int, data: EnumValueCreate, db: Session =
Depends(get_db)):
    """Обновить значение перечисления"""
    result = classifier.update_enum_value(db, value_id, data.value, data.sort_order,
data.extra_data)
    if not result["success"]:
        raise HTTPException(status_code=400, detail=result["message"])
    return result

@app.put("/enumerations/{enum_id}/values/reorder",
response_model=OperationResult)
def reorder_enum_values(enum_id: int, data: EnumValueReorder, db: Session =
Depends(get_db)):
    """Изменить порядок значений перечисления"""
    result = classifier.reorder_enum_values(db, enum_id, data.ordered_ids)
    if not result["success"]:
        raise HTTPException(status_code=400, detail=result["message"])
    return result

@app.delete("/enum-values/{value_id}", response_model=OperationResult)
def delete_enum_value(value_id: int, db: Session = Depends(get_db)):
    """Удалить значение перечисления"""

```

```

result = classifier.delete_enum_value(db, value_id)
if not result["success"]:
    raise HTTPException(status_code=400, detail=result["message"])
return result

@app.post("/reset-database")
def reset_database():
    """Полностью пересоздаёт базу данных (только для разработки!)"""
    import psycopg2
    from config import config

    try:
        conn = psycopg2.connect(
            host=config.DB_HOST,
            port=config.DB_PORT,
            user=config.DB_USER,
            password=config.DB_PASSWORD,
            database="postgres"
        )
        conn.autocommit = True
        cursor = conn.cursor()

        # Закрываем все соединения
        cursor.execute(f"""
            SELECT pg_terminate_backend(pg_stat_activity.pid)
            FROM pg_stat_activity
            WHERE pg_stat_activity.datname = '{config.DB_NAME}'
            AND pid <> pg_backend_pid()
        """)

```

```
cursor.execute(f'DROP DATABASE IF EXISTS {config.DB_NAME}')
cursor.execute(f'CREATE DATABASE {config.DB_NAME}')

cursor.close()
conn.close()

return {"success": True, "message": f'База данных {config.DB_NAME}
пересоздана. Перезапустите приложение."}
except Exception as e:
    return {"success": False, "message": str(e)}
```